

GAUTAM BUDDHA UNIVERSITY
IT CELL INTERN RECRUITMENT — SCREENING CHALLENGE 2026

GrievEase Bot

*An AI-Powered Conversational Assistant for the
GBU Grievance Management & Ticket Tracking System*

Submitted for the role of:

AI Chatbot Development & Deployment (Experienced Intern)

Applicant Name: [Your Full Name]

Enrollment/Roll No.: [Your Roll Number]

Contact: [Your Email] | [Your Phone Number]

Date: July 2026

Table of Contents

1. Executive Summary
2. Problem Understanding
3. Proposed Solution Overview
4. Chatbot Architecture
5. Conversation Flow Design
6. Prompt Design
7. Knowledge Base Structure
8. Integration with the Grievance Management System
9. Technology Stack
10. Key Features
11. Additional Innovations
12. Security, Privacy & Scalability
13. Evaluation Criteria Mapping
14. Deliverables & Project Roadmap
15. Conclusion

1. Executive Summary

GrievEase Bot is a conversational AI assistant designed to sit on top of the GBU Grievance Management & Ticket Tracking System and act as the primary, always-available interface between students and the grievance redressal process. Instead of students having to navigate a form and separately remember to check a portal for updates, the chatbot allows them to submit a grievance, track its status, and get answers to process-related questions through a simple, guided conversation — in natural language, at any hour, from any device.

The bot is designed around three principles: it should be easy to talk to, it should never leave a student without an answer (falling back to human escalation when unsure), and it should integrate cleanly with the core grievance system rather than duplicating it. This document covers the complete design: architecture, conversation flow, prompt design, knowledge base structure, technology choices, security considerations, and the additional innovations proposed beyond the mandatory requirements.

2. Problem Understanding

The core system to be built is a GBU Grievance Management & Ticket Tracking System where students submit grievances, receive an auto-generated Ticket ID, and track resolution status (Pending → In Progress → Resolved), with an Admin Dashboard for staff and mandatory email notifications at key stages.

As the AI Chatbot intern, the responsibility is not to rebuild this system but to design an intelligent conversational layer over it that:

- Lowers the barrier to submitting a grievance — many students abandon multi-field forms; a conversational flow feels less intimidating and can be completed in under a minute.
- Removes the need to "remember to check back" — students can simply ask the bot for their ticket status instead of navigating back to a portal and searching.
- Reduces repetitive load on the Admin/IT Cell by answering common process questions (e.g. "How long does resolution take?", "Can I edit my grievance after submitting?") automatically.
- Knows when it doesn't know — a chatbot that confidently gives wrong information about a grievance is worse than no chatbot; escalation to a human must be a first-class design decision, not an afterthought.

The key design challenge, therefore, is building a system that is conversationally natural but operationally precise — it must reliably capture structured data (category, description, ticket ID) from unstructured chat input, and must integrate correctly with the underlying ticketing APIs without silently dropping or corrupting information.

3. Proposed Solution Overview

GrievEase Bot is proposed as a hybrid rule-based + LLM-powered chatbot (not a purely free-form LLM chat, and not a rigid decision-tree bot — a middle ground that combines the reliability of structured flows for critical actions with the flexibility of an LLM for natural conversation and FAQ handling).

3.1 Why hybrid, not purely LLM-driven?

Grievance submission and status tracking are data-critical operations — a wrong category or a mistyped ticket ID has real consequences for a student. A purely generative approach risks hallucinating ticket statuses or misfiling information. GrievEase Bot therefore uses:

- A deterministic, slot-filling dialogue flow for the two mandatory actions — Submit Grievance and Track Status — so every required field is validated before an API call is made.
- An LLM + Retrieval-Augmented Generation (RAG) layer for open-ended questions (FAQs, guidance, clarifications) where flexibility matters more than rigid structure.

- A confidence-based router that decides, per user message, whether the input matches a known intent/flow or should be handled by the LLM/FAQ layer or escalated to a human.

3.2 High-level capability map

Capability	Required by Challenge	How GrievEase Bot Delivers It
Assist in submitting grievances	Yes	Guided multi-turn form-filling via chat, with confirmation before submission
Track grievance status via Ticket ID	Yes	Direct API call to Status endpoint, natural-language status summary
Answer grievance-related FAQs	Yes	RAG-based retrieval over a structured Knowledge Base
Guide users through the process	Yes	Proactive menu, contextual hints, and fallback suggestions
Escalate to a human when needed	Innovation	Confidence threshold + sentiment/urgency detection routes to Admin
Multi-lingual support (Hindi/English)	Innovation	Language auto-detection with Hinglish handling

4. Chatbot Architecture

The architecture is layered so that each concern — channel handling, orchestration, reasoning, and data integration — can be built, tested, and scaled independently. This also means the LLM/reasoning layer can later be swapped (e.g. a different model or provider) without touching the grievance system's core APIs.

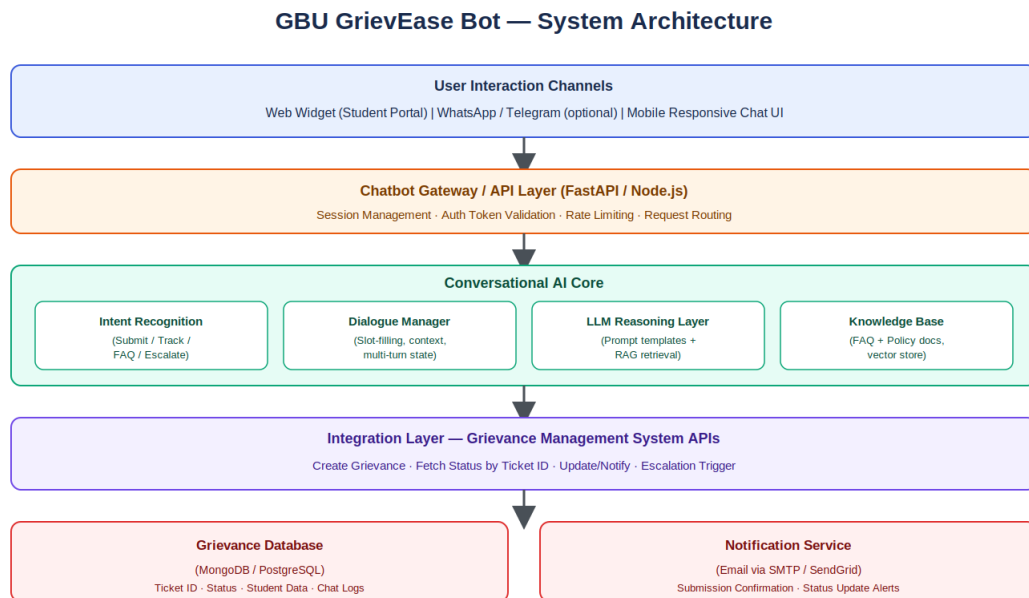


Fig 1: End-to-end architecture — the chatbot sits as an intelligent front layer over the Grievance Management System's core APIs.

Fig 1: End-to-end architecture — the chatbot sits as an intelligent front layer over the Grievance Management System.

4.1 Layer Breakdown

- **Channels:** Web chat widget embedded in the student portal (primary), with the architecture designed to extend to WhatsApp/Telegram later without redesign.
- **Gateway/API Layer:** Handles session identity, authentication token validation (so the bot knows which student it's talking to), request routing, and basic rate limiting to prevent abuse.

- **Conversational AI Core:** The brain of the system — intent recognition, a dialogue manager that tracks conversation state (e.g. "we're 2 of 4 fields into a submission"), and the LLM reasoning layer that handles free-form questions using the knowledge base.
- **Integration Layer:** A thin, well-defined API contract to the Grievance Management System (create grievance, get status, list categories) — this is the only place the chatbot talks to the core system, keeping coupling minimal.
- **Data & Notification:** The grievance database itself (owned by the core system) and the notification service that sends the two mandatory emails (Grievance Submitted, Status Updated).

5. Conversation Flow Design

The conversation is designed around four core intents that map directly to the challenge's required capabilities, with a consistent entry menu and a graceful exit/escalation path so the user is never stuck.

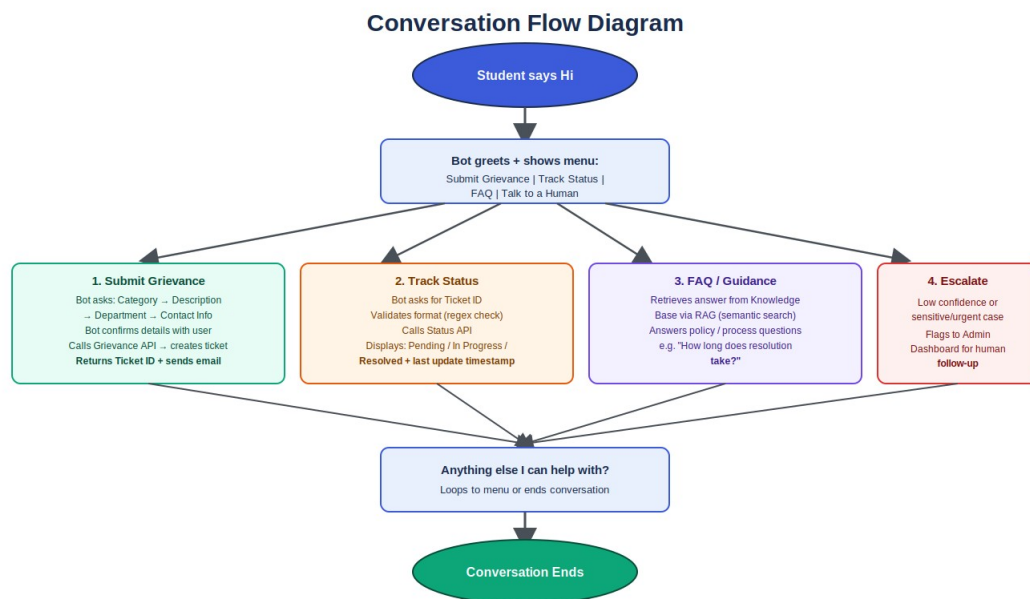


Fig 2: Multi-intent conversation flow with graceful fallback to human escalation.

Fig 2: Multi-intent conversation flow with graceful fallback to human escalation.

5.1 Sample Dialogue — Submitting a Grievance

Student: "My hostel wifi has not worked for a week"

Bot: "I'm sorry to hear that. I can log this as a grievance for you. I'll file it under Hostel/IT Infrastructure — is that correct?"

Student: "Yes"

Bot: "Got it. Could you share your hostel block and room number so the technical team can locate the issue?"

Bot: "Thanks — here's a summary: Category: Hostel/IT Infrastructure, Issue: Wi-Fi not working for 7 days, Location: Block C, Room 214. Shall I submit this?" → "Your grievance has been submitted. Your Ticket ID is GBU-2026-004821. You'll get an email confirmation, and you can ask me for updates anytime using this ID."

5.2 Sample Dialogue — Tracking Status

Student: "What's the status of GBU-2026-004821?"

Bot: "Your grievance GBU-2026-004821 (Hostel Wi-Fi issue) is currently In Progress. It was assigned to the IT Infrastructure team on 22 July and is expected to be resolved within 3 working days."

6. Prompt Design

Prompt design follows a layered system-prompt strategy: a fixed system prompt defines the bot's persona, boundaries, and output rules; a dynamic context block injects retrieved knowledge-base snippets and conversation state; and per-intent instruction templates guide structured tasks like slot extraction.

6.1 Core System Prompt (structure)

Section	Purpose	Example Content
Persona	Sets tone and scope	"You are GrievEase Bot, GBU's grievance assistant. Be concise, empathetic, and factual."
Capabilities	Defines what it can do	"You can help submit grievances, check status by Ticket ID, and answer FAQs about the grievance process."
Guardrails	Prevents overreach	"Never invent a ticket status. If an API call fails or returns no data, say so and offer escalation."
Output format	Ensures consistency	"Keep responses under 4 sentences unless summarizing a submission."
Escalation rule	Safety net	"If confidence is low or the student sounds distressed/urgent, offer to connect them with a human staff member."

6.2 Intent-Specific Prompt Templates

- Slot-extraction prompt: given the raw student message, extract structured fields (category, description, location) as JSON, explicitly instructed to leave a field null rather than guess if not mentioned.
- FAQ/RAG prompt: "Using only the knowledge base context below, answer the student's question in 2–3 sentences. If the answer isn't in the context, say you're not sure and offer to escalate. Context: {retrieved_chunks} Question: {user_question}"
- Confirmation prompt: reformats collected slots into a plain-language summary and asks for explicit yes/no confirmation before any API write — this single step prevents most submission errors.

6.3 Few-Shot Examples for Reliability

A small set of curated few-shot examples (3–5 per intent) are included in the prompt for category classification and slot extraction, covering common grievance types (Hostel, Academic, Examination, IT/Wi-Fi, Faculty, Administrative) so the model reliably maps informal student language to the correct fixed category list used by the Admin Dashboard.

7. Knowledge Base Structure

The Knowledge Base (KB) powers the FAQ and guidance capability through Retrieval-Augmented Generation: relevant chunks are retrieved via semantic search and passed to the LLM as grounding context, rather than relying on the model's own (potentially outdated or invented) knowledge of GBU-specific policy.

7.1 KB Content Categories

Category	Example Content
Process FAQs	"How long does resolution typically take?", "Can I withdraw a submitted grievance?", "What happens after I submit?"

Category	Example Content
Category Definitions	Descriptions of each grievance category (Academic, Hostel, Examination, IT, Administrative, Ragging/Harassment) so the bot classifies correctly
Escalation Policy	Which categories are auto-flagged as sensitive/urgent (e.g. harassment) and routed straight to human staff, bypassing bot handling
Contact Directory	Relevant department contacts for issues outside the grievance system's scope
Sample Resolved Cases (anonymized)	Used to calibrate expected response times per category shown to students

7.2 Structure & Storage

- Each KB entry is stored as a short, self-contained chunk (100–200 words) with metadata tags: category, audience, last-updated date.
- Chunks are embedded (vector representation) and stored in a lightweight vector store (e.g. FAISS/Chroma) for semantic similarity search.
- A versioned JSON/Markdown source-of-truth is maintained by the Admin team, re-embedded whenever content changes — keeping the KB easy to update without touching bot code.

8. Integration with the Grievance Management System

The chatbot communicates with the core system exclusively through a small, well-documented set of REST endpoints, keeping the two systems loosely coupled:

Endpoint	Method	Purpose
/api/grievance	POST	Create a new grievance; returns generated Ticket ID
/api/grievance/{ticketId}	GET	Fetch current status, category, and last-updated timestamp
/api/grievance/categories	GET	Fetch the current list of valid categories (kept in sync with Admin Dashboard)
/api/notify	POST	Trigger email notification (submission confirmation / status update)

This contract-based approach means the Web Development (Experienced) team's backend and the chatbot can be developed in parallel against an agreed API spec, and either side can be modified independently as long as the contract holds.

9. Technology Stack

Layer	Proposed Technology	Reasoning
Chat Widget (Frontend)	React + Tailwind CSS	Lightweight, embeddable, consistent with a responsive portal
Bot Backend / Gateway	Python (FastAPI)	Fast to prototype, strong AI/ML ecosystem, async-friendly
LLM Reasoning	Claude API (Anthropic)	Strong instruction-following for structured slot extraction + safe, grounded responses

Layer	Proposed Technology	Reasoning
RAG / Vector Store	Chroma or FAISS	Lightweight, open-source, easy to self-host for a university project
Database	MongoDB (or shared with core system's DB)	Flexible schema for chat logs; can align with core system's choice
Notifications	SMTP / SendGrid	Reliable transactional email delivery
Deployment	Render / Railway (or GBU server) + Docker	Simple containerized deployment within a short timeline

10. Key Features

1. Conversational grievance submission — guided, multi-turn, with an explicit confirmation step before final submission.
2. Ticket status lookup by natural language — students can ask "what's happening with my ticket" instead of remembering exact portal navigation.
3. FAQ & process guidance via RAG — grounded answers pulled from a maintained knowledge base, not free-form model guesses.
4. Category auto-classification — maps informal descriptions to the fixed category list used by the Admin Dashboard.
5. Confidence-based human escalation — any low-confidence or sensitive case (e.g. harassment-related) is flagged for a human, never silently handled by the bot.
6. Mandatory email notifications — triggered by the bot on submission and forwarded from the core system on status change.
7. Responsive, embeddable widget — works on both desktop and mobile views of the student portal.

11. Additional Innovations

Beyond the mandatory deliverables, the following enhancements are proposed to strengthen usability, efficiency, and overall experience — aligned with the challenge's encouragement of creative, value-adding features:

- Bilingual support (English + Hindi/Hinglish): auto-detects the language a student is typing in and responds in kind, since GBU serves a linguistically diverse student body.
- Sentiment & urgency detection: flags grievances with distressed or urgent language (e.g. safety concerns) for immediate human review, rather than treating every ticket with equal priority.
- Proactive status nudges: optionally, the bot can offer "Would you like me to notify you the moment this changes status?" instead of the student needing to ask again.
- Duplicate-grievance detection: before creating a new ticket, the bot checks if the student has an open, similar grievance and asks if they'd like to add to the existing ticket instead of creating a duplicate.
- Feedback capture on resolution: when a ticket moves to Resolved, the bot can follow up with a 1-tap satisfaction rating, feeding a simple analytics view for the IT Cell.
- Admin-side chat analytics: a lightweight summary (most common grievance categories, average bot-handled vs. escalated ratio) to help the IT Cell spot recurring campus issues.

12. Security, Privacy & Scalability

12.1 Security

- Authentication: the bot only creates or fetches grievances tied to an authenticated student session token — it never accepts a raw student ID typed in chat as sufficient identity proof for fetching another student's data.
- Input validation: all slot values are validated/sanitized server-side before any database write, regardless of what the LLM extracted, to prevent injection or malformed data.
- Rate limiting: per-session request throttling at the gateway layer to prevent abuse or automated spam submissions.
- Least-privilege API access: the chatbot's service account has only the specific grievance-create/read/notify permissions it needs — not full database access.

12.2 Privacy

- Chat logs containing personal grievance details are stored with the same access controls as the core grievance database, not in a separate, less-protected log store.
- Sensitive categories (e.g. harassment) are never used as few-shot/training examples or exposed in analytics in identifiable form.

12.3 Scalability

- Stateless gateway design allows horizontal scaling of the bot backend independent of the core grievance system.
- The KB/RAG layer scales by adding documents without retraining any model — new policies can be added by simply updating the knowledge base.
- Clear API contract (Section 8) means the chatbot and core system can be scaled, deployed, and even re-hosted independently.

13. Evaluation Criteria Mapping

Evaluation Criterion	Where Addressed in This Document
Problem Understanding	Section 2 — explicit breakdown of the grievance system's needs and the chatbot's specific role within it
Creativity & Innovation	Section 11 — bilingual support, sentiment detection, duplicate detection, feedback loop, analytics
Technical Approach	Sections 4, 6, 7, 8 — architecture, prompt design, KB structure, and API integration contract
Documentation Quality	This document — structured, diagrammed, and cross-referenced end-to-end
Practical Feasibility	Section 9 — realistic, buildable tech stack suited to a short timeline
Overall Presentation	Cover page, diagrams (Figs. 1–2), tables, and consistent formatting throughout

14. Deliverables & Project Roadmap

This documentation forms Day 1 of a 3-day build plan for the screening submission:

Day	Deliverable	Contents
Day 1 (this	Solution Idea + Full	Problem understanding, architecture, conversation

Day	Deliverable	Contents
document)	Documentation	flow, prompt design, KB structure, tech stack, innovations
Day 2	Source Code + GitHub Repository	Working FastAPI backend, RAG pipeline, mock grievance API, structured repo with README
Day 3	Prototype + Demo Link	Deployed chat widget with end-to-end submit/track/FAQ flow, short walkthrough recording

14.1 Submission Checklist (per the application form)

- Brief Project Description (150–200 words) — see condensed summary prepared separately for the form field.
- GitHub Repository Link — to be added after Day 2.
- Live Demo / Deployment Link — to be added after Day 3.
- Prototype / Figma Link — optional; a working chat prototype will be prioritized over static mockups given the "Experienced" track.
- Project Documentation (this PDF) — this file.

15. Conclusion

GrievEase Bot is designed to be more than a FAQ chatbot bolted onto a form — it is a structured, hybrid conversational layer that makes grievance submission and tracking genuinely easier for students while staying disciplined about accuracy, security, and graceful escalation. The architecture keeps the chatbot loosely coupled to the core Grievance Management System via a clear API contract, making it realistic to build, test, and deploy within the 3-day timeline while leaving clear room for the additional innovations described in Section 11 to be layered in incrementally.

The next step (Day 2) is implementing this design end-to-end: the FastAPI gateway, the intent/dialogue logic, a working RAG pipeline over a seed knowledge base, and a mock or shared grievance API to submit and track real tickets against.